

01. Learning to correct without breaking everything else

00:00-01:00

Our question is simple to state: can a system learn a correction without breaking things it already does? The longer-term ambition is an agent that can also decide when to intervene, when to hold back, and what to investigate next. That brings together the three subjects of the conference session: active inference, predictive coding, and heavy-tailed distributions.

Today I want to show you enough of the machinery and the actual outputs to judge the story for yourselves. This is the longer colleague version of a talk I will give in Binghamton on October 15. As we go, please remember one example or result you would keep in a fifteen-minute version, and one place where the explanation loses you. I will pause twice for quick reactions and leave the broader discussion until the end.

Sources: [brief](#), [short_deck](#)

02. Three questions, three different kinds of evidence

01:00-02:15

Here is the route through the talk. Active inference asks what an agent should do next, given what it believes, what it wants, and what it might learn. Predictive coding asks how prediction errors can guide updates within a system. Heavy-tail analysis asks about the distribution of consequences, especially uncommon large ones that a mean can hide.

Our evidence is at different stages for those three questions. We have a working correction testbed. We have experiments that change the learning rule and measure the resulting trade-offs. We also have saved probability distributions that let us examine rare harmful changes. The autonomous action-selection loop remains a proposal. Making that distinction helps us say what we have learned without treating the motivation as a completed result. I will first make the testbed concrete, then show the extremes, then the predictive-coding results, and finally reconnect them to the proposed agent.

Sources: [brief](#), [mapping](#)

03. Active inference starts with a loop

02:15-03:55

Imagine an agent maintaining beliefs about a changing world. An observation can lead it to revise those beliefs. An action can change the world or reveal information about it. Some outcomes matter more than others, so preferences belong in the model as well. Active inference brings those pieces into a common framework for inference and policy selection. The proposal compares policies by expected free energy, with preferred consequences and information value under an explicit probability model. Prediction-error learning can be one component, but it does not by itself supply the whole loop.

In our setting, someone supplies a correction. The system can store it and decide whether to use it when a new question arrives. A future agent might additionally decide which question to test next: another paraphrase, a nearby fact, or an ordinary-text context where a correction could do damage. That action would buy information at some cost. The proposed policy would need an explicit probability model, preferences, and a way to compare information value with consequences. We have not implemented that policy. The experiments I am about to show provide a place to ask whether it would be useful.

Sources: [brief](#), [mapping](#), [proposal](#)

04. The cap changes an activation, not the frozen base weights

03:55-05:35

A language model maps the text so far into probabilities for the next token. Its internal activations pass through a sequence of transformer blocks. We keep the main model's learned weights fixed, and place an adaptive correction system, which we call a cap, around a few of those internal sites.

The cap stores records for the supplied facts. A trained reader compares the current query with memory, and a gate can choose no correction at all. When a record is selected, bounded residual vectors are added at the write sites. The model then continues computing its next-token distribution. A residual is simply an added correction to an existing activation; we are not replacing the entire transformer.

The attraction is a separation between a stable prior and adaptable memory. The difficulty is that a small local intervention can still have consequences downstream. Freezing the weights prevents one kind of change; it does not guarantee unchanged predictions. The diagrams in our abstract call these two interfaces blankets. Here they are software interfaces; the required statistical blanket properties have not been demonstrated.

Sources: mapping, stage4

05. One edit has several tests to pass

05:35-07:00

An edit begins with a prompt and the answer we want the system to produce. First we ask whether it can produce that answer immediately. After a long stream of other edits, we ask the original prompt again. That tests whether the memory survived. Then we ask a paraphrase, a differently worded version of the same request. That tests whether the system can find and use the memory outside the exact teaching prompt.

Paraphrase retention is the main outcome in the central study.

We also ask about preservation. Unrelated prompts should keep their old behavior. Nearby prompts are more challenging because they resemble an edited question while referring to another fact. Revisions test whether the newer answer replaces an older one. Finally, ordinary text lets us compare token probabilities even when a visible answer does not change. These tests will often disagree, and that disagreement is part of the result rather than a nuisance to average away.

Sources: [stage4](#), [examples_readme](#)

06. A recorded correction, after 1,000 edits

07:00-08:50

Here is an actual saved example, not an invented demonstration. The question asks which league Sporting Canamy was in. The supplied target is Tercera División de México. The cap-free model emits a newline immediately, so its bounded answer is empty. All three displayed cap conditions acquire the taught answer.

Now move to the end of the thousand-edit stream. The learned and random-reader conditions still answer the original question correctly, but the stable cap's bounded answer is empty. Rephrase the question, and the learned reader still supplies the target. The random reader supplies the name of a different club, while the stable cap says forward. That is a concrete difference between storing a correction and retrieving the right correction when the wording changes.

This is the first item of one realization and one order, selected to make the mechanisms understandable. It is an old memory by the endpoint, and it is not a representative sample of the entire experiment. The aggregate results come later. I have removed leading display spaces from these quotations; the source file retains the complete generated strings and scoring flags.

Sources: examples

07. A memory can survive while its paraphrase fails

08:50–10:40

CounterFact asks a slightly different question. We deliberately teach a counterfactual target, so the target is an experimental instruction, not a claim about the world. In the first example we teach Spanish for James Howell, where the dataset lists English as the previously true answer. The paraphrase includes an unrelated prefix sentence, yet the learned cap produces Spanish.

The second example teaches Philippines for the origin of Gol and Gincu The Series, where the stored previously true answer is Malaysia. At the endpoint, the original prompt still gets Philippines. But the paraphrase receives the displayed free continuation instead. The saved selection flag says that the learned cap did not fire on this paraphrase. We can therefore describe the observed routing failure without guessing from the final score alone.

The contrast is useful because both memories look successful if we only repeat the teaching prompt. The paraphrase reveals the difference. It also reminds us that these are strict bounded-answer scores: producing fluent text is not the same as giving the requested experimental target.

Sources: examples

08. Preserving behavior can mean preserving a bad answer

10:40–12:20

Here is a nearby-fact test. We teach New Zealand for the displayed Frits Poelman prompt, then ask the similar question about George Pitt Morison. The base's bounded answer is empty. The learned reader keeps it empty; the random reader instead supplies New Zealand, the supplied target for the other question. A third condition produces Paris. The dataset's stored neighboring answer is Australia, but preservation is scored against the base's output, not against that stored factual answer.

The same issue is especially clear in an unrelated locality prompt about Fred Flintstone's wife. The base outputs a question mark and the learned cap also outputs a question mark. That passes preservation; it is plainly not a demonstration of answering the question correctly. This is why I will not call a perfect locality score a guarantee of safe or knowledgeable behavior.

Pause for about twenty seconds: does the actual input-and-output example make the correction task clearer than the architecture diagram? A quick show of hands is enough; save longer reactions for the discussion.

Sources: [examples](#), [stage4](#)

09. What the main comparison holds fixed

12:20-13:50

The central comparison puts a learned reader beside a random-geometry reader with a gate and an older stable cap. The main-condition transformer is frozen GPT-2 small. There are additional controls, but these three make the main question easiest to see: how much useful retrieval comes from learning the reader rather than just having writable memory?

We use three datasets with different prompting styles. The zsRE task uses questions. CounterFact uses counterfactual sentence completions and paraphrases. MQuAKE contributes single-hop edits from its data; our experiment is not a test of its full multi-hop reasoning task. The first two reach a thousand edits; MQuAKE is reported descriptively at three hundred.

Three separately drawn subject realizations are each run in five orders. Those orders can reveal order sensitivity, but they reuse facts. Later reader-training seeds also reuse one exposed subject realization. Those are useful controlled interventions, with a narrower generalization claim. I will mark the different populations rather than combine all the runs as though they were one larger independent experiment.

Sources: stage4, triplet

10. The learned reader retains more paraphrases

13:50-15:25

Each small point is a subject realization after averaging its five orders. The large marks show the mean of the three realizations. Keeping those points visible is more informative here than showing a large sample size built from repeated questions or tokens.

The learned-reader result is strongest on zsRE, with about ninety-six percent paraphrase retention. CounterFact is about sixty-eight percent. MQuAKE is about seventy-two percent, at its separate three-hundred-edit endpoint. These numbers are scores on the supplied targets, not a general knowledge or reasoning benchmark. The graph also shows how much the comparisons depend on the dataset.

The useful result is that the learned reader can carry many corrections beyond the exact teaching wording. The next question is what changed elsewhere in the model's predictions. The main study's limited-realization inference is preliminary; this graph deliberately shows descriptive realization spread, not a new confidence interval. A later fourth-realization check against the random reader is available in the backup slides, without rewriting the original analysis.

Sources: triplet, stage4

11. A changed probability can reveal harm before the answer changes

15:25-17:15

To see unintended changes, take a piece of ordinary text and stop at a particular prefix. We know the next token in that text. Compare the probability assigned to it by the base with the probability assigned by the capped model. Negative log probability is the loss, so our loss difference is the logarithm of base probability divided by capped probability. A positive value means the cap made that observed token less likely. Negative values are improvements.

The arithmetic example on screen is illustrative: a drop from ten percent to one percent gives $\log_{10} 10$, about two point three nats of increased loss. One nat corresponds to a probability reduction by a factor of about two point seven. Neither example is a newly measured case.

KL asks a related but different question: how much did the whole next-token distribution change? All forty-five learned-reader cells in the main triplet exceeded the registered mean-KL benchmark of one thousandth. Their integrity checks passed, meaning the experimental records are usable. That does not mean the behavioral fidelity requirement passed. We need both efficacy and preservation to understand the result.

Sources: ht17, stage4

12. How often something goes wrong is not how badly it goes wrong

17:15-18:55

The left panel shows the fraction of positions where loss rises by more than one hundredth of a nat. The right panel asks how large the increase is among those positions. For the learned reader, the frequencies are about zero point sixteen percent on zsRE and zero point thirty percent on CounterFact. The corresponding conditional severities are about one point six and one point nine nats.

The random reader on CounterFact is worse on both displayed measures. The stable cap on CounterFact has no observed harm in this inventory, but it also has zero paraphrase retention. Quietness alone is therefore not our goal. On zsRE, different summaries give a less uniform ranking: the learned reader can improve editing while increasing some harm summaries relative to stable memory.

Every cell reuses the same ordinary-text inventory. The positions are dependent, and reusing them does not multiply the number of independent subjects. These plotted values describe this fixed inventory. The full report retains signed improvements, zeros, maxima and expected shortfalls as well as the two quantities shown here.

Sources: tails, ht17

13. Rare large errors motivate tail analysis; they do not prove a tail law

18:55–20:40

A survival curve answers: what fraction of observations exceed this size? Reading farther to the right focuses on larger consequences. In a heavy-tailed model, sufficiently large observations die away more slowly than an exponential tail. But seeing a few large observations is not enough to establish that mathematical property for the underlying system.

We fitted excess distributions above declared thresholds, checked how many windows supplied the events, and asked whether the more flexible generalized-Pareto model predicted held-out windows better than an exponential. For the learned reader, the extra shape parameter added little predictive value. The stable cap on zsRE showed a more pronounced tail over the measured range. Some negative-shape fits for the random reader failed because they assigned zero probability to held-out extremes beyond their fitted endpoint. That is a useful warning against treating a fitted upper bound as a guarantee.

The plotted empirical curves use the same preselected realization and order. They illustrate the measured range. We have not established an asymptotic power law, infinite variance, a thermodynamic temperature or a complexity class. The important connection to the session is learning to inspect extremes rather than assuming that the mean tells the whole story.

Sources: [tails](#), [ht17](#)

14. A simple mixture gives a real per-token bound

20:40–22:15

One intervention has a guarantee we can derive rather than fit. Mix the base model's next-token distribution with the cap's distribution. Keep about thirty-seven percent of the base and sixty-three percent of the cap. Because probabilities are nonnegative, every token keeps at least that base-weighted floor. Taking the log ratio gives a maximum loss increase of one nat relative to the base, at that same prefix.

This does not require the empirical tail to follow a particular family. It is a property of the mixture. It also permits large probability increases for desired tokens; it only prevents their probabilities from falling too far relative to the base floor. There is a trade-off because mixing can change the greedy answer, so retention still has to be measured.

The guarantee is specifically about a token probability comparison at a common prefix. It is not a one-nat guarantee for a whole answer, and it does not establish that KL falls below our much smaller benchmark. Once generated prefixes diverge, one must be careful about what comparison is being made.

Sources: awb, awb_data

15. The mixture reduced severity while largely retaining the edits

22:15–23:55

We selected the mixture using development data and then evaluated it on ten exposed memories: two datasets and five orders, at three hundred edits. The graph shows mean severity among positions with loss increase above one hundredth of a nat. On zsRE it falls from about one point six two to zero point five four nats. On CounterFact it falls from about one point nine three to zero point five eight. That is roughly one-third of the original severity, not a reduction of only one-third.

All ten memories meet the specified retention and preservation rule. The frequency of harmful changes barely moves. CounterFact paraphrase retention has a small cost, and its mean KL still exceeds the benchmark. We also tried a kappa-based bounded-surprisal training loss. That pilot missed its declared success rule. It was not the complete coupled-entropy or coupled-free-energy objective proposed in the theoretical programme.

The distinction matters: one result is a useful analytic bound with a measured trade-off; the other is a negative pilot on a particular loss. Neither is a general verdict on coupled active inference.

Sources: tails, awb, pilot

16. Predictive coding: infer errors before choosing an update

23:55–26:00

Now to the learning rule. A taught target creates a prediction loss. We need a signal saying which internal correction would reduce that loss. The adjoint path computes a derivative through the model, then uses the negative normalized direction. This is our comparison condition.

In the error-inference path, we introduce temporary error variables at the write sites. Start them at zero. Hold the existing memory writes fixed while those error variables settle. The energy balances a quadratic penalty on the inferred errors against the task loss produced with those errors present. After a finite number of optimization steps, the inferred site errors supply directions for the same bounded memory update used by the comparison arm. Our default is eight settling steps at step size zero point one.

That is the predictive-coding component we tested. There are different predictive-coding formulations; this implementation uses automatic differentiation through the computation graph. It is not evidence for biologically local learning or an implementation free of backpropagation. The scientific question is whether this particular iterative credit rule buys more useful correction, less harm, or a better trade-off for its cost. Those are measured questions, not consequences of calling the method predictive coding. Separately, we also trained the reader itself using error inference; I will distinguish that experiment from changing acquisition credit.

Sources: `pc_spec`, `pc0`, `pc1`

17. A repaired energy and a useful one-step sanity check

26:00–27:20

An earlier error in the PC implementation penalized the inferred error plus the existing write, rather than penalizing the inferred error alone. That changes the objective when a write is already present. We corrected it before the supplemental comparisons shown here. The main feedforward reader experiment did not use this defective credit path.

There is a simple mechanism check. At zero inferred error, the quadratic penalty has zero derivative. The first gradient step is therefore minus the step size times the adjoint. Our common normalization removes that positive scale, so the first-step directions agree in exact arithmetic. In the actual one-step runs, all recorded behavioral endpoints agree, while small floating-point differences remain in some harm vectors. This makes one step an informative baseline. Any benefit from deeper settling must earn its extra cost; it cannot be assumed from the presence of an iterative algorithm.

Sources: [pc_spec](#), [controls](#), [stage4](#)

18. More settling bought memory retention, with costs

27:20–29:00

These are the same zsRE subjects and the same order at depths one, eight and thirty-two, averaged over three realizations. On the left, own-prompt retention rises with deeper settling. Paraphrase retention does not show the same gain and falls at thirty-two steps. The other panels show the cost: expected shortfall of positive harm rises, and learning takes much longer.

Expected shortfall here means the average of the worst one percent of positive-part loss values across the whole inventory, including zeros. It is not the ninety-ninth percentile and not a mean over only the positive observations. This older base uses a smaller ordinary-text inventory, so do not compare its absolute tail values with the main GPT-2 study as if only the learning rule changed.

The main eight-step comparison across all five orders found about a two-percentage-point own-prompt gain on zsRE, with no consistent paraphrase gain. Random-direction credit mostly failed in its one attempted stream. The extra-update adjoint control was offered the PC budget but stopped early, so it does not settle the question of credit direction versus equal realized compute. Those details are in backup.

Sources: `depth`, `controls`, `pc_spec`

19. Changing credit on the working reader gave small, mixed effects

29:00–30:20

We next moved the acquisition-credit comparison onto the working GPT-2 reader. The base, trained reader, gate, memory capacity and teaching streams stayed fixed; each arm acquired a fresh memory. At three hundred edits, zsRE paraphrase retention is unchanged. CounterFact falls by about a third of a percentage point with error credit. The positive-harm expected shortfall improves a little on zsRE and worsens a little on CounterFact.

The acquisition-and-assay stream totals rise from about six hundred five seconds to seven hundred seventy-seven seconds across the two datasets. The separate full harm readout took about five thousand seconds for the comparison. I would not describe these results as a clear win or as proof that the algorithms are equivalent. This is a one-realization exposed transfer check. It tells us that changing this credit rule alone did not produce a compelling new paraphrase benefit in the tested working reader.

Sources: pc1

20. Training the reader with PC was much more expensive

30:20–32:05

This experiment changes a different part of the system. We trained the reader and controller with backpropagation or with error inference, using paired initializations and the same training episodes. Acquisition used adjoint credit in both conditions. The left panel shows the difference in paraphrase retention for each training seed, with error inference minus backpropagation as the sign convention.

On CounterFact, every seed is lower with the PC-trained reader: by about twenty-four, two and seventeen and a half percentage points. zsRE has two small deficits and one small gain. The right panel shows the measured training-time ratio: roughly one hundred times as much process time for error inference. Each PC training took about a day, compared with roughly fifteen minutes for BP.

The harm results do not provide a uniform compensation: CounterFact harm directions change across seeds. Some zsRE PC readers are very quiet, but quietness can accompany missed useful corrections. Three paired training seeds help reveal that variability; they are not three independent subject populations. This is a substantive negative result for this implementation and recipe. It does not rule out other PC architectures, tuning choices, or learning formulations.

Sources: reader, reader_doc

21. Restricting the cap to upper layers did not remove the harm

32:05–33:25

We also tested the idea that the cap might benefit from using only higher transformer layers. The design crosses two choices: read all three taps or only the upper two, and write at all three sites or only the last one. There are three paired seeds and two datasets, for twenty-four evaluations, including shared BP controls.

The graph isolates the write comparison. Every pair has more mean ordinary-text loss with last-site-only writes, by about two point six to four point four times. Paraphrase scores barely change in those pairs. Reading only upper taps also lowers CounterFact paraphrase retention in all three seeds when writes use all sites. This does not support the assumption that the lower interfaces are mostly noise.

There is a scope limit: these readers were trained with a full-write objective. Last-only writes were evaluated with newly acquired memories under that restriction, not with a separately optimized last-only training objective. A differently trained architecture remains a future question.

Sources: [upper](#), [upper_doc](#)

22. The next agent could choose which failure to investigate

33:25–35:15

Now return to the agent loop. We have examples of successful correction, failures to retrieve, unintended changes to nearby facts, and rare losses on ordinary text. Today the investigators choose which of those tests to run. An active-inference extension could choose an audit because it expects that observation to resolve uncertainty or avoid a costly intervention.

For example, it might have budget for one additional query. It could repeat the teaching question, ask a new paraphrase, or test a neighboring fact. Which gives the most useful information about whether the stored correction should be used? The concrete proposed comparison is an explicit policy against fixed audit schedules, under the same resources, measuring correction utility and harm together.

The feedback from the coupled-entropy work points to a further modeling task: specify the modeled random variable, the probability distribution, constraints, weighting and gradients before claiming a coupled free-energy objective. Our bounded kappa loss did not do that. The measured mixture is a useful baseline, and tail severity gives us a consequence to model, but neither constructs the policy for us. This is a post-conference direction, not a promise of another experiment before October ninth.

Sources: proposal, mapping, feedback

23. The experimental portfolio is closed; the story is still being refined

35:15–36:20

The planned GPU work is now done, or explicitly closed where resource limits prevented completion. The main snapshot contains two hundred seventy completed cells. Predictive-coding credit studies, the three-seed reader comparison, the mixture intervention and the upper-layer experiment have their own reports. Tail analysis uses the saved outputs.

The original experimental deadline is October ninth at five p.m. Eastern. Finishing now leaves time to review the fixed evidence, make the slides intelligible, and rehearse before October fifteenth. The main snapshot alone used three hundred ninety-two process-hours. That is summed worker time, not elapsed GPU time; concurrent workers overlap.

The most useful remaining work for this talk is not collecting another impressive number. It is checking that the figures and examples support the interpretation, making the scope understandable, and deciding which parts deserve the limited conference time. Your reactions today help with that last part.

Sources: [stage4](#), [reader_doc](#), [upper_doc](#), [decisions](#)

24. Three takeaways to carry into the conference

36:20–37:25

Here are the three points I currently want a conference audience to remember. First, the frozen-prior architecture supplies concrete components for an active-inference programme, while autonomous policy choice is still a missing piece. Second, predictive coding was tested as an actual intervention in the learning machinery. We found trade-offs and substantial costs, rather than a general advantage for the tested recipe. Third, rare harmful changes make distributional analysis essential: frequency, severity and maxima can tell different stories. A simple probability mixture provides a useful bounded baseline.

The successful corrections are real, the negative results are useful, and the limits shape the next experiment. That combination seems more informative than treating either the architecture or the theoretical framework as already validated. I would like your help deciding which examples make that message most memorable and which technical details are necessary to trust it.

Sources: [stage4](#), [pc1](#), [reader_doc](#), [ht17](#), [proposal](#)

25. Help choose the fifteen-minute story

37:25-39:25

Before we open the discussion, take about forty-five seconds to write down three slide numbers. Which one would you keep if I had time for only one detailed example? Which needs a clearer explanation? And which would you move to the question period? It is also useful to tell me what claim sounded stronger than the evidence actually shown.

I am especially interested in whether the concrete answer examples, the predictive-coding mechanism, or the distributional harm story gives you the strongest way into this project. I am not asking you to pick the most positive result. A negative result that teaches something important may be the right center of the talk. The short conference version must still connect all three themes, but the depth can follow what you find worth discussing.

After the writing pause, invite two brief reactions, then open the wider discussion. The remaining slides are optional backup on MQuAKE, scoring, PC controls, the fourth-realization check and sources.

Sources:

26. MQuAKE: a single-hop editing example

Optional backup

This optional example shows Marion Brown taught the counterfactual target West Coast hip hop, with jazz as the stored old answer. The learned cap supplies the target on the original prompt and its paraphrase at three hundred edits. A second example about Bettino Ricasoli keeps Methodism on the original prompt but produces an empty answer on the paraphrase, even though the stored selection flag says the cap fired. That differs from the CounterFact no-fire example. We cannot conclude that every paraphrase failure has the same cause. We also cannot claim multi-hop reasoning: this test uses the single-hop edit and paraphrase portion of the task. These selected examples do not estimate a success rate.

Sources: examples, stage4

27. The risk summaries answer different questions

Optional backup

Delta loss compares the observed next token at a common prefix. KL compares the whole predictive distribution, in the reference-to-cap direction. Signed mean loss retains improvements as negative values. Conditional severity uses only values above the specified threshold. Expected shortfall uses the positive part over the full inventory, including zero values. We sort that inventory and take the fractional mean of its worst one percent. Because one percent of 245,237 is not an integer, the boundary observation gets fractional weight. A maximum is a different statistic again. Inferences over windows condition on the tested models and subject populations; they do not cover training or subject variation.

Sources: ht17, pc1

28. What the extra PC controls do and do not settle

Optional backup

The random-direction control retains the norm and pays for the true error-credit calculation before replacing the direction. Its only attempted random stream stopped at 993 edits under the resource rule, with three immediate successes. Ten planned cells were unstarted. It supports the practical importance of informative direction on that stream, not a universal impossibility result about random search.

For the compute control, charlie approved funding additional acquisition updates rather than repeated derivative calculations at unchanged writes. The adjoint condition was offered the measured per-item PC operation allowance. It used only about fifty-three to fifty-eight percent, because its stopping rules usually ended acquisition sooner. This does not isolate extra computation as a cause of the PC effect, nor is it a FLOP-matched experiment. The six complete pairs and the partial random experiment are reported separately.

Sources: controls, matched

29. A fourth subject draw supports the learned-random contrast

Optional backup

The fourth-realization extension completed twenty learned-reader and random-reader cells, at a thousand edits, across five orders and two datasets. The new-realization paraphrase means are shown here. Combining four realization-level contrasts gives a learned-minus-random mean of about forty-four percentage points on zsRE and fifty-six on CounterFact. The supplemental t intervals assume normal realization errors and are unadjusted; they are not a replacement familywise analysis. The own-prompt contrasts are negative, so the generalization gain still needs to be distinguished from literal prompt retention. One stable-cap attempt was incomplete and nine such cells were deferred. Those missing comparisons are not zeroes.

Sources: option_r

30. Source map and the submitted programme

Optional backup

The submitted title is Coupled Active Inference on a Frozen Transformer Prior: A Risk-Aware Residual Agent for Non-Equilibrium Regimes, by charlie derr and Matthew Iklé. The main report and its triplet analysis support the main comparison. The supplemental reports distinguish the acquisition-credit, reader-training, mixture and upper-layer experiments. Capstan's support-information collection provides the saved input and output examples. Our post-conference proposal identifies the unresolved modeling choices for a coupled-objective and audit-policy collaboration. This talk is a snapshot of completed results through October fourth; earlier documents that say a result is pending are historical.

Sources: brief, stage4, ht17, reader_doc, examples_readme, proposal